

Lab 2: Max detection
Geavanceerde computerarchitectuur

(ULTRA-B-KUL-JLI1QS-2324)

Michiel Van Nieuwenhove

February 25, 2024



Group: MEIswe

Instructor: Alex Marinšek

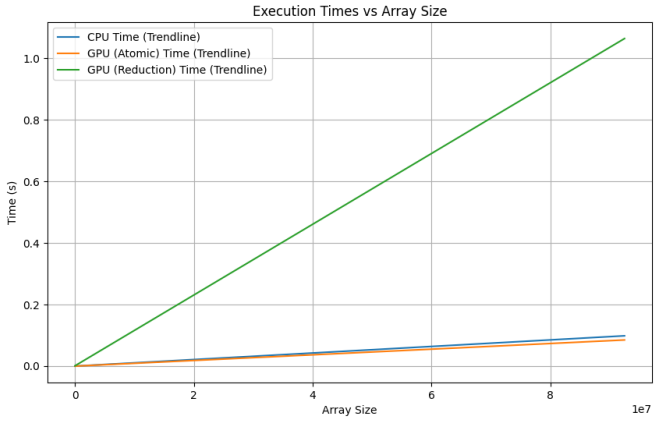
1 Implementation

The first way to implement this, with a nested for-loop, makes it so the array gets looped through sequentially. Whenever a new maximum value is found, this gets stored to compare with the next values. This happens completely on the CPU, so no checking of different cells in the array happens in parallel.

To implement this wtih an atomic operation, the 'atomicMax' function is used. Because of its atomic nature it ensures that the read-modify-write operation is performed without interruption from other threads. This means that even if multiple threads attempt to execute atomicMax concurrently on the same memory address, the updates will be performed in a serialized manner, ensuring consistency.

To implement this with reduction, a function for one block at a time is defined. This function assignes threads to compare two values. The maximum value gets inserted back into the array. For example, the array [0, 5, 6, 3] gets two threads, one to compare 0 and 5, and one to compare 6 and 3. The resulting array will be: [5, 6, 6, 3]. The last two values are not important anymore. This step will be executed over and over again with a smaller number of elements, but with less and less threads, untill the amount of elements to be compared is 1. this is the maximum value. If the amount of cells in the array is too big for one block, the array is divided in multiple ones, and the function is called sequentially. This avoids race conditions. If the array is not of size 2^N, the algorithm will come to a point where it has to compare the last cell in the array with a cell outside of its bounds. At this point the algorithm will carry the element to the next step without performing an operation on it, so at the end it will be taken into account.

2 Evaluation & Conclusion



The nested for-loop seems to be the most performant for small array-sizes. For large array-sizes, the implementation with 'atomicMax' performs the best. The implementation with reduction performs way worse than the others for all instances. It seems that the time gained by using reduction is not enough to nullify the performance-loss by using blocks sequentially, to avoid race conditions between them. All algorithms give the same result, so race conditions are avoided.